

AWS Deployment Guide — Nucleus AI

This guide covers deploying Nucleus AI on AWS using **ECS Fargate** (serverless containers) or **EC2 with Docker Compose**.

Prerequisites

- AWS CLI v2 installed and configured (`aws configure`)
 - Docker installed locally
 - An AWS account with appropriate IAM permissions
 - A domain name pointed to AWS (Route 53 or external DNS)
-

Option A: ECS Fargate (Recommended for Production)

1. Create ECR Repositories

```
# Create repositories for backend and frontend images
aws ecr create-repository --repository-name nucleus-ai/backend --region us-east-1
aws ecr create-repository --repository-name nucleus-ai/frontend --region us-east-1

# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.us-
east-1.amazonaws.com
```

2. Build and Push Images

```
# Build production images
docker build -f docker/production/Dockerfile.backend -t nucleus-ai/backend .
docker build -f docker/production/Dockerfile.frontend \
  --build-arg NEXT_PUBLIC_API_URL=https://api.yourdomain.com \
  -t nucleus-ai/frontend .

# Tag and push
docker tag nucleus-ai/backend:latest <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/
nucleus-ai/backend:latest
docker tag nucleus-ai/frontend:latest <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/
nucleus-ai/frontend:latest

docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/nucleus-ai/backend:latest
docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/nucleus-ai/frontend:latest
```

3. Set Up RDS PostgreSQL

```
# Create a VPC-private RDS PostgreSQL instance
aws rds create-db-instance \
  --db-instance-identifier nucleus-db \
  --db-instance-class db.t3.medium \
  --engine postgres \
  --engine-version 16.4 \
  --master-username nucleus \
  --master-user-password "YOUR_STRONG_PASSWORD" \
  --allocated-storage 20 \
  --storage-type gp3 \
  --vpc-security-group-ids sg-XXXXX \
  --db-subnet-group-name your-subnet-group \
  --backup-retention-period 7 \
  --multi-az \
  --storage-encrypted
```

Important: Enable automated backups, encryption at rest, and place the RDS instance in a private subnet.

4. Set Up Qdrant

Option 1: Qdrant Cloud (Recommended)

- Sign up at cloud.qdrant.io (<https://cloud.qdrant.io>)
- Create a cluster in your preferred AWS region
- Note the URL and API key

Option 2: Self-hosted on ECS

- Create an ECS service for `qdrant/qdrant:v1.12.5`
- Attach an EFS volume for persistent storage
- Configure security group to allow port 6333 only from backend tasks

5. Store Secrets in AWS Secrets Manager

```
aws secretsmanager create-secret \
  --name nucleus-ai/production \
  --secret-string '{
    "POSTGRES_PASSWORD": "YOUR_STRONG_PASSWORD",
    "OPENAI_API_KEY": "sk-...",
    "NEXTAUTH_SECRET": "...",
    "SECRET_KEY": "..."
  }'
```

6. Create ECS Cluster and Task Definitions

```
# Create cluster
aws ecs create-cluster --cluster-name nucleus-ai-prod

# Register task definitions (see task-definition examples below)
aws ecs register-task-definition --cli-input-json file://ecs-backend-task.json
aws ecs register-task-definition --cli-input-json file://ecs-frontend-task.json
```

Backend Task Definition (`ecs-backend-task.json`):

```

{
  "family": "nucleus-backend",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "executionRoleArn": "arn:aws:iam::ACCOUNT:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::ACCOUNT:role/ecsTaskRole",
  "containerDefinitions": [
    {
      "name": "backend",
      "image": "<ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/nucleus-ai/
backend:latest",
      "portMappings": [{"containerPort": 8000, "protocol": "tcp"}],
      "environment": [
        {"name": "APP_ENV", "value": "production"},
        {"name": "DEBUG", "value": "false"},
        {"name": "DATABASE_URL", "value": "postgresql+asyncpg://nucleus:PASS@rds-host:
5432/nucleus"},
        {"name": "QDRANT_URL", "value": "https://your-qdrant-cluster.cloud.qdrant.io"}
      ],
      "secrets": [
        {"name": "OPENAI_API_KEY", "valueFrom": "arn:aws:secretsmanager:us-
east-1:ACCOUNT:secret:nucleus-ai/production:OPENAI_API_KEY::"},
        {"name": "SECRET_KEY", "valueFrom": "arn:aws:secretsmanager:us-
east-1:ACCOUNT:secret:nucleus-ai/production:SECRET_KEY::"}
      ],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"],
        "interval": 30,
        "timeout": 10,
        "retries": 3,
        "startPeriod": 15
      },
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/nucleus-backend",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "backend"
        }
      }
    }
  ]
}

```

7. Create Application Load Balancer

```
# Create ALB
aws elbv2 create-load-balancer \
  --name nucleus-ai-alb \
  --subnets subnet-XXX subnet-YYY \
  --security-groups sg-XXX \
  --scheme internet-facing

# Create target groups
aws elbv2 create-target-group --name nucleus-backend --port 8000 --protocol HTTP \
  --target-type ip --vpc-id vpc-XXX --health-check-path /health
aws elbv2 create-target-group --name nucleus-frontend --port 3000 --protocol HTTP \
  --target-type ip --vpc-id vpc-XXX --health-check-path /api/health

# Create HTTPS listener with ACM certificate
aws elbv2 create-listener --load-balancer-arn <ALB_ARN> \
  --port 443 --protocol HTTPS \
  --certificates CertificateArn=arn:aws:acm:us-east-1:ACCOUNT:certificate/XXX \
  --default-actions Type=forward,TargetGroupArn=<FRONTEND_TG_ARN>

# Add rule: /api/* → backend target group
aws elbv2 create-rule --listener-arn <LISTENER_ARN> \
  --conditions Field=path-pattern,Values='/api/*' \
  --actions Type=forward,TargetGroupArn=<BACKEND_TG_ARN> \
  --priority 10
```

8. Create ECS Services

```
aws ecs create-service \
  --cluster nucleus-ai-prod \
  --service-name nucleus-backend \
  --task-definition nucleus-backend \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-XXX],securityGroups=[sg-XXX],assignPublicIp=DISABLED}" \
  --load-balancers "target-GroupArn=<BACKEND_TG_ARN>,containerName=backend,containerPort=8000"

aws ecs create-service \
  --cluster nucleus-ai-prod \
  --service-name nucleus-frontend \
  --task-definition nucleus-frontend \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-XXX],securityGroups=[sg-XXX],assignPublicIp=DISABLED}" \
  --load-balancers "target-GroupArn=<FRONTEND_TG_ARN>,containerName=frontend,containerPort=3000"
```

9. Auto Scaling

```
# Register scalable targets
aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/nucleus-ai-prod/nucleus-backend \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 10

# CPU-based scaling policy
aws application-autoscaling put-scaling-policy \
  --service-namespace ecs \
  --resource-id service/nucleus-ai-prod/nucleus-backend \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-name cpu-scaling \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "TargetValue": 70,
    "PredefinedMetricSpecification": {"PredefinedMetricType": "ECSServiceAverageCPUUtilization"},
    "ScaleInCooldown": 300,
    "ScaleOutCooldown": 60
  }'
```

Option B: EC2 with Docker Compose

Best for smaller deployments or teams that prefer simpler infrastructure.

1. Launch EC2 Instance

- **AMI:** Ubuntu 24.04 LTS
- **Instance type:** t3.large (2 vCPU, 8 GB RAM) minimum
- **Storage:** 50 GB gp3 EBS
- **Security Group:** Allow ports 80, 443, 22

2. Server Setup

```
# SSH into instance
ssh -i your-key.pem ubuntu@<EC2_PUBLIC_IP>

# Install Docker
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker ubuntu

# Install Docker Compose
sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-com-
pose-$(uname -s)-$(uname -m)" \
  -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose

# Clone your repository
git clone https://github.com/your-org/nucleus-ai.git /opt/nucleus-ai
cd /opt/nucleus-ai

# Create production env file
cp .env.production.example .env.production
nano .env.production # Fill in real values

# Set up SSL with Let's Encrypt
sudo apt install certbot
sudo certbot certonly --standalone -d yourdomain.com -d www.yourdomain.com
sudo cp /etc/letsencrypt/live/yourdomain.com/fullchain.pem nginx/ssl/
sudo cp /etc/letsencrypt/live/yourdomain.com/privkey.pem nginx/ssl/

# Deploy
docker compose -f docker-compose.prod.yml --env-file .env.production up -d
```

3. Set Up Certbot Auto-Renewal

```
echo "0 0 1 * * certbot renew --quiet && docker compose -f /opt/nucleus-ai/docker-
compose.prod.yml restart nginx" \
| sudo crontab -
```

Monitoring on AWS

CloudWatch

- ECS tasks automatically send logs to CloudWatch
- Create CloudWatch dashboards for CPU, memory, and request metrics
- Set up CloudWatch Alarms for error rates and high CPU

Cost Estimation (Monthly)

Service	Spec	Estimated Cost
ECS Fargate (Backend x2)	1 vCPU, 2 GB	~\$60
ECS Fargate (Frontend x2)	0.5 vCPU, 1 GB	~\$30
RDS PostgreSQL	db.t3.medium, Multi-AZ	~\$70
ALB	Standard	~\$20
ECR	Image storage	~\$5
Qdrant Cloud	Starter	~\$25
Total		~\$210/mo

EC2 Option: A single t3.large running everything costs ~\$60/mo + RDS.

Security Checklist for AWS

- ☐ RDS in private subnet (no public access)
- ☐ ECS tasks in private subnets with NAT Gateway
- ☐ Security groups: least-privilege rules
- ☐ Secrets in AWS Secrets Manager (not env vars)
- ☐ Enable RDS encryption at rest
- ☐ Enable ALB access logging to S3
- ☐ Enable AWS WAF on ALB
- ☐ Enable CloudTrail for audit logging
- ☐ Use IAM roles (not access keys) for ECS tasks